Research toward fewer than 208 states for arithmetic consistency

The 207 state target remains unmet

Research prepared for Vladimir Reshetnikov

29 October 2026

Abstract

The goal of constructing a binary blank-input machine with fewer than 208 working states has not been met. No validated 207-state transition table or proof is claimed. Five incoming archives contain nine literal 208-state tables but exactly two rooted transition-graph classes. All use the same conditional history fold of two native states, 172 and 211; their reductions cannot be stacked to obtain 207 states. Fresh runtime replays cover both classes. This note explains the common fold, separates its finite simulation proof from the inherited consistency calibration, and records further source and local-machine searches. A conditional pruning removes five source primitives but increases the present syntax DAG from 197 to 200 nodes. New seven-cell runtime projections strengthen the search beyond recent-transition history, without yielding a validated smaller machine. Negative results concern explicitly bounded search families, not a lower bound on state complexity or an exhaustive search of machines.

1 The target and what has been established

The requested object is a deterministic binary Turing machine M with at most 207 working states, including its complete blank initializer, such that

$$\mathsf{PA} \vdash (\neg\,\mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{U})).$$

The convention is one head, one two-way infinite tape, alphabet $\{0, 1\}$, blank symbol 0, and only left and right moves. The halt label is separate and is not counted as a working state. The initial head is at position 0 and all tape cells are zero. Native labels in a sparse table are names, not the count; in the Joint native labeling used below, 211 is a working label and 254 is halt.

No validated machine meeting the target is supplied here. The established input level remains 208 states and 416 total binary rows. The positive results of this investigation are an exact comparison of all nine supplied tables, fresh runtime replays for their two graph classes, and precisely scoped reusable proof and search evidence. These do not turn a 208-state table into a 207-state construction.

The distinction between a PA-provable equivalence and actual nonhalting is important. A correct consistency calibration does not require assuming $\mathrm{Con}(\mathsf{U})$ in order to prove the biconditional. Asserting that the machine actually runs forever requires the corresponding consistency assumption. Nor is an ordinary PA-formalizable argument automatically a newly exported proof-assistant certificate. The latter is not claimed in this report.

2 Five archives and exactly two graph classes

The initial intake is pinned to ProveIt commit `ef5a23e04b0285e321ceec5f4b329223cee0ab57`; the later intake is pinned to `34aaab66fc5dc834863ed00ea84823c93b8c5b1e`. Both are snapshots of `docs/incoming`. Archive SHA-256 values and Git blob identities are retained in the comparison packet. The table below names the archives without their common date suffix, 2026-10-29.

Archive	Tables	Graph classes
`U208_ConPA_Conditional_Context`	2	Primary, Compatibility
`U208_ConPA_Entry_Context`	2	Primary, Compatibility
`U208_ConPA_History`	2	Primary, Compatibility
`U208_ConPA` (late)	1	Primary
`u208_research` (late)	2	Compatibility, Primary

The Primary tables in Conditional Context and Entry Context are byte-identical; their Compatibility tables are also byte-identical. The corresponding canonical native-table digests are

Primary

`b348a466a2c316511e0037ea3824de29d4c6ad37338404fbc98a3968fdb25135`

Compatibility

`5f936da0cf393548cdd1eeebcaf7490d7a42f4ccca115f865a7bcc045463eb52`

History retains the other representative of the merged 172/211 roles and uses other labels. The late submissions also use other native labels. These are not additional structural reductions: exact graph conjugacies identify each with its appropriate representative.

For a table δ, a conjugacy g is a bijection of working states and halt, fixing the distinguished initial and halt states, such that

$$\delta(q, b) = (w, d, r) \quad \Longrightarrow \quad \delta'(g(q), b) = (w, d, g(r)).$$

Induction on time then transports every computation on every common initial tape, preserving the head, all tape cells, and the number of steps. The checker verifies this equation on all 416 rows of every table. To distinguish the two classes, it also constructs the canonical rooted deterministic traversal, visiting read-0 before read-1 successors. The canonical tables differ; this is an exact nonisomorphism test, not a failed heuristic. Nonisomorphism alone does not assert a difference in computed behavior.

All nine snapshots have 208 graph-reachable working states. Graph reachability is weaker than reachability on the single blank run, so it is not a minimality argument. All source snapshots have the same ordered 3,909 primitives and the same 5,083 guarded ordinal successor arrows. Two differently structured source resolvers agree. The Primary source has depth 42; the alternative layout has depth 41. Source word and control-flow identity do not imply identical binary addresses, physical decoding paths, or literal-machine step counts.

3 The common history fold and its invariant

Let A be the Joint 209-state parent and B its Primary 208-state reduction. The reduction deletes state 172, redirects its only entrance, and changes one retained transition. In the notation wDr for write, direction, and target,

Row	Parent A	Reduced B
116/0	$0R172$	$0R211$
211/0	$0R157$	$0L217$
172/0	$0L67$	deleted
172/1	$1R184$	deleted

Every other retained row is unchanged. Define $f(172) = 211$ and $f(q) = q$ for all other labels, including halt. At simulation boundaries, f changes only the state; it leaves the tape and head unchanged.

For a configuration (q, h, T) write $t_j = T(h + j)$. The relevant parent invariant consists of the following implications from the indicated state:

$$\begin{array}{r|l} 116 & t_{-1} = 1 \\ 172 & t_{-2} = 1 \ \wedge \ t_{-1} = 0 \\ 217 & t_{-1} = 1 \\ 163 & t_{-1} = 1 \ \vee \ t_{-2} = 1 \\ 211 & t_0 = 1 \ \vee \ t_{-1} = 1. \end{array}$$

In particular, 211 reading zero implies a one immediately to the left. It does not imply that every visit to 211 has a left-hand one. The complete incoming census in the parent is

Target	All incoming physical rows
172	$116/0 : 0R172$
116	$215/1 : 1R116$
211	$163/0 : 0L211$, $163/1 : 1R211$
163	$132/1 : 1R163$, $199/1 : 1R163$, $217/0 : 0R163$
217	$216/1 : 1R217$

Lemma 3.1. *Every parent configuration reached from state 0 satisfies these five implications, on any initial binary tape.*

Proof. Initially they are vacuous. An entrance to 116 or 217 writes one and moves right. An entrance to 172 writes zero and moves right from 116, carrying its left-hand one to offset -2. An entrance to 163 either writes a left-hand one or, from 217/0, carries such a one to offset -2. Finally, 163/1 establishes a left-hand one at 211; 163/0 moves left, so the former two left cells become the new current and left cells. Their disjunction gives the condition at 211. These are all entrances, so induction proves the claim. □

The census includes physical engine entrances, not merely edges of the source syntax. This is why the proof applies to arbitrary initial tapes and not only to an observed blank prefix or a selection of register values.

4 Whole tape simulation and why the folds cannot stack

Only old states 172 and 211 need multi-step treatment. The four replacement contracts in B are

Old state	Read	Required context	Steps	Endpoint $(q, h - h_0)$
172	0	$t_{-2} = 1,\ t_{-1} = 0$	7	$(67, -1)$
172	1	none beyond read	1	$(184, +1)$
211	0	$t_{-1} = 1$	3	$(157, +1)$
211	1	none beyond read	1	$(184, +1)$

Every block preserves the entire tape. Both one-cases use $211/1 : 1R184$. The three-step zero-case follows $211 \to 217 \to 215 \to 157$ at relative head positions $0, -1, 0, 1$, reading and rewriting $0, 1, 0$. The seven-step zero-case has this complete trace:

Step	State before	Head before	Read and write	State after
1	211	0	0	217
2	217	−1	0	163
3	163	0	0	211
4	211	−1	0	217
5	217	−2	1	215
6	215	−1	0	157
7	157	0	0	67

The final head is at -1. No unspecified cell is inspected, every write preserves its cell, and no internal state is halt. The other 414 parent rows commute with f in one step. In particular, the internal return to state 211 in this seven-step block does not invoke the parent invariant afresh; the explicit trace supplies the proof there.

Theorem 4.1. *The parent and reduced machines, started in state 0 on the same tape and head, halt together. If $N_{q,b}(n)$ counts parent transitions of type (q, b) among the first n steps, they agree in tape and head, and in state after applying f, at reduced time*

$$\tau(n) = n + 6N_{172,0}(n) + 2N_{211,0}(n), \qquad n \le \tau(n) \le 7n.$$

Proof. Induct over parent steps using the invariant and the four contracts. Their positive lengths make τ unbounded on an infinite parent run, and the successive blocks cover every reduced time. None halts internally. Thus a parent first halt gives exactly the corresponding reduced first halt, while an infinite parent computation gives an infinite reduced computation. □

In Compatibility native labels, the selector 217 becomes 214, the feeder 163 becomes 216, and the right branch 215 becomes 163; the same proof applies. History keeps the opposite merged representative. These are alternative presentations of the same one-state saving, not two deletions that survive simultaneously. The nine-table graph audit makes this precise. The finite invariant, traces, and bounded clock give a PA-formalizable halting equivalence; they do not independently establish the inherited consistency endpoint.

5 The consistency endpoint and the verification boundary

The theory U is equality-free classical first-order logic over a nonempty domain with one binary relation E. Write $\mathrm{Sub}(x,a) := \forall t(E(t,x) \to E(t,a))$. For every formula φ with free variables contained in $\{x\}$, its weak-cover schema is

$$\forall a\, \exists b\, [\forall x(E(x,b) \to \varphi(x)) \ \wedge\ \forall x(\mathrm{Sub}(x,a) \to (\varphi(x) \to E(x,b)))]\,.$$

Bound variables are freshened. The source a is not allowed as a free matrix parameter. A weak cover may contain additional selected objects, so one cannot silently replace this by an exact subset filter.

The supplied calibration argues that U and PA admit uniform finite-proof translations verifiable in PA. Its lower interpretation uses coextensional numerical classes, parameter internalization, numerical induction, bounded comprehension with explicit support, and deterministic histories for arithmetic. The upper interpretation is finite binary membership: $E(x,a)$ means that bit x of a is one. For each fixed formula, PA induction supplies the required finite characteristic code, without a uniform truth predicate.

The native source does *not* literally enumerate only unary U instances. Its machine-facing theory is unrestricted weak cover W plus irreflexivity IR. The retained proof transport gives the cycle

$$\mathrm{Con}(\mathsf{PA}) \Longrightarrow \mathrm{Con}(\mathsf{W}+\mathsf{IR}) \Longrightarrow \mathrm{Con}(\mathsf{U}) \Longrightarrow \mathrm{Con}(\mathsf{PA}).$$

The numerical axiom calculus, proof compiler, unbounded scheduler completeness, and protected startup are separate obligations in that chain. In particular, the initializer reaches an interior axiom-command suffix with an opaque stack tail. The inherited proof builds nine theorem heads and maintains a permanent margin of seven above that tail; a finite trace by itself would not establish that the tail is never exposed.

The current audit freshly replays the ordinary symbolic runtime engine over arbitrary natural register values, arbitrary gap, and finite spare-register lists, not just small samples. Its two representative receipts are

Class	Primitive contracts	Covered rows	Symbolic inferences
Primary	5,084	416	1,251,865
Compatibility	5,084	416	1,250,564

The exact blank startup takes 48,140 steps, ending at native state 84, head -2, source ordinal 136, with logical registers $(1,0,13,0,0,1,0,0,1)$. The Primary full replay also checks 104 Boolean invariant cases and rejects 12 negative controls. The History release's quick replay and late-copy conjugacies are not misreported as additional fresh full runtime proofs.

Composing the literal simulation with the *inherited* numerical and theory calibration yields the supplied 208-state endpoint $\mathsf{PA} \vdash \neg\,\mathrm{Halt}(B,\mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{U})$. This audit is not a fresh kernel proof of those logical dependencies. In particular, a separate K215 formalization concerning ZFC cannot be relabeled as a theorem for these tables. The local proof is complete on its stated premises; the full consistency calibration remains an explicit dependency.

6 Conditional source pruning and the sharing obstruction

The source lane investigates deleting two closed loops whose bodies are unreachable under a previously checked source invariant. In old zero-based ordinals, the guards are $R_1 = 0$ at 3847 and $R_6 = 0$ at 3891. Conditional on those guards, bodies 3848–3849 and 3892 are never entered. The pruning packet checks the following exact tree edits:

Case	Replace subtree by its right child	Deleted interval
First	$001101110 \mapsto 0011011101$	$[3847, 3850)$
Second	$1110011 \mapsto 11100111$	$[3891, 3893)$

The first removes a closed transfer loop; the second removes a closed clearing loop. The numerical names of ordinals are not physical machine-state labels.

Proposition 6.1. *Assuming the two zero guards at every reachable loop entry, the two-loop pruning is a source-level stuttering simulation in both directions, with unchanged retained registers and halt behavior.*

Proof. The external-entry census has only 3841/zero $\to$ 3847 and 3889/zero $\to$ 3891. There is no outside entrance to a deleted body. At either guarded loop entry, the conditional-decrement step sees zero, changes no register, and immediately exits to retained ordinal 3850 or 3893. Thus each deleted reachable segment is exactly one stuttering step, not an unbounded hidden computation. The initial ordinal 136 is retained.

Delete the indicated intervals and map each retained old ordinal to its new rank. Bypass any old edge into a deleted interval to that interval's retained successor. Two independent source-control resolvers check that every new retained edge is exactly the transported old edge with this bypass, including both decrement branches. Induction matches every retained boundary in either direction; the possible inserted segments are finite and nonhalting. The halt occurrence remains retained. This establishes the claimed correspondence. □

The portable checker verifies syntax, both control-flow resolutions, every retained edge, the external-entry census, and the following metrics. It does not reprove the zero-guard invariant or certify a new binary interpreter.

Source	Primitive occurrences	Distinct syntax DAG nodes	Depth
Original	3,909	197	42
First deletion only	3,906	198	42
Second deletion only	3,907	199	42
Both deletions	3,904	200	42

Maximum zero-run length stays six. Removing executed syntax does not necessarily improve subtree sharing: local edits can destroy repeated factors. In the current binary table, relevant decoder entries 19/28 and 103/108 also serve many retained source occurrences. Their global deletion is therefore not justified by the two local guards. These DAG counts describe one syntax and are not Turing-state lower bounds. A different representation or compiler could behave differently; no 208-to-207 conclusion follows from this pruning.

7 Further source layouts and local synthesis

Closed reassociation searches check all 3,908 internal source-prefix bindings and admit decoding segments within 50 steps. A pure right-reading middle factor excludes a known retagging collision. The guarded lanes restrict edited decoder labels; the wider lane removes that restriction. The fold scanner uses opposite predecessor tags, a common branch, and existing two-cell helpers within 40 steps; two lanes also test duplicate rows.

Layout lane	Layers	Beam	Additional tables
Primary guarded	12	16	1,003
Compatibility guarded and duplicates	12	16	1,141
Primary wider and duplicates	8	16	2,358

All use seed 13 and have zero additional fold survivors. Their counts overlap; they are not numbers of nonisomorphic machines. Any survivor would still need engine, runtime, and startup proofs. Separately, an eight-layer, beam-eight, seed-seven reassociation search after both source deletions visits 10,218 additional syntax trees and never improves on 200 DAG nodes. It emits no binary machine and proves no DAG lower bound.

History guards and complete tape macro contracts

Backward analysis retains all consistent incoming histories, including paths from state 0 shorter than the depth bound. Unrecorded tape cells remain arbitrary. These guards hold from state 0 on any initial tape; intersecting them with a forward window invariant initialized blank restricts the claim to blank executions. A candidate macro must preserve the full old tape effect, head, mapped successor, positive progress, and absence of an interior halt. Unknown cells are symbolically branched where permitted; concrete test tapes are rejection filters, not success certificates.

Depth-12 shared-row searches check 148 pairs and 122,544 actions per class, with a 60-step cap, finding no fold. The neutral catalog has 351 Primary and 357 Compatibility row identities. Fresh replay checks all 708 identities via 42,240 history-context checks and 24,987 consistent symbolic branches. Composing them with folds yields 3,138,357 and 3,159,139 helper-repair attempts, with no accepted result.

Two-row join family at radius four and cap 32	Tested pairs	Actions
Primary	21,321	19,100,685
Compatibility	21,321	19,111,897
Primary with source-derived guards	17,493	15,388,630

The last lane shrinks windows at 119 states and skips 3,828 unchanged pairs already covered by the Primary search. Its raw index 21,321 is not its tested pair count. All three runs are negative. Initial-control-inclusive joins also check 207 pairs and 192,663 actions per class, with no survivor.

Selectors and head offset aliases

Two deletions paid for by one new selector give 214 conditional-history pair combinations and 443 broader driver-macro combinations across the two classes, without a witness. Offset aliases at $k = \pm 2$ test 74,106 noninitial choices. Adding one recorded neutral helper gives 2,001,827 trials, while 24,224,006 further choices are excluded by first divergence on a failing bounded trace. Initial-state offset variants test 15,592 candidates. None passes its necessary finite 40-step screen. These failures reject bounded sufficient contracts, not blank-run equivalence; admitted test contexts may themselves be unreachable. The offset simulation principle below is separate from these observations.

8 Runtime derived windows and their proof scope

A different context source projects the full symbolic tape before every literal inference in the 5,084 primitive contracts, together with every configuration of the exact blank startup. For radius three, a window records the cells at offsets $-3, \ldots, 3$, with the least significant bit representing -3. This brings source-boundary information into the local search rather than relying solely on a short backward transition history.

Projection deliberately forgets correlations. A nonconstant nonnegative affine run length $c + \sum_i a_i x_i$ permits all lengths at least c; a constant run has its exact length. Only the requested prefix width is relevant. Opaque atoms and finite-list atoms permit every bit prefix of that width. The infinite blank exterior contributes only zeros; left and right choices are combined independently. Every such relaxation enlarges the set of possible windows and is therefore safe for proving a local identity on all of them.

A compressed unary or list traversal can cross configurations absent from the literal-inference endpoints. The conservative projection assigns every window to states used in such opaque traversals, including literal states occurring in a proved list block. It does not infer internal invariants from endpoint summaries. The specialized lane retains this conservative treatment and tightens only the source-register categories justified by a separate invariant.

Lemma 8.1 (Coverage of the projected windows)**.** *Assume the inherited source semantics and full-frame symbolic runtime contracts. Every working-state configuration of the intended blank computation has its seven-cell window in the set recorded for that state. The specialized sets have the same property when the checked source-category invariant is also used.*

Proof. The entire finite startup and its endpoint are explicitly included. Thereafter source semantics decomposes execution into primitive contracts. At a literal inference, each concrete affine-run length and atom realization is among the projection's permitted choices, and discarding correlations can only enlarge them. Orientation is explicit: the left prefix has length three or four, according to which stream holds the current cell, and is reversed before concatenation with the right prefix. Zero-length runs retain their empty case. Every internal state of an opaque compressed traversal receives all windows. A contract endpoint is the next complete source frame or halt. Induction over contracts therefore covers all intermediate times. Specialization restricts only stores and branches already excluded by the source-category invariant, so the same proof applies to every reachable store. □

The universal projection records 7,697 state-window pairs over 5,084 primitive contracts. The source-category specialization records 7,694 pairs over 5,079 contracts, removing exactly three windows. Both retain all 128 seven-bit windows for each of the 25 conservatively opaque engine states. Specialization uses the separately checked product-category invariant at source ordinals, transported to the exact ordered control-flow graph. It does not assume that an arbitrary register store is reachable.

Window search family	Pair or branch choices	Actions	Macro cap
Universal common-row folds	224	186,368	30
Specialized common-row folds	224	186,368	30
Universal generalized branch folds	489	406,848	30

All three completed runs have zero accepted witnesses. The first two require one row to agree after quotient; the generalized family permits the unchanged branch to agree by a macro. The 30-step cap is a proof-search restriction, not a semantic runtime bound. A ten-stage full replay of the boundary bundle passes. In each Python mode, finite projection sanity checks cover 2,880 instances and reject eight tampered controls; these tests supplement, rather than replace, the all-length projection argument.

9 Reproduction and the remaining constructive obligation

No completed bounded test provides a validated 207-state machine. Failure can reflect the representation, overapproximation, edit family, or cutoff. It is not an exhaustive search or a lower bound.

9.1 Reproducible audit layers

At the archive root, run

```
python3 -B reproduce.py --receipt /tmp/below-u208-replay.json
```

The default performs full replay; `--quick` explicitly skips expensive runtime, projection, and synthesis components. Component checks differ in scope:

1. In `comparison`, run `python3 -B check.py` and `python3 -B -O check.py`. Both verify all nine table hashes, totality, graph census, conjugacies, two canonical graph classes, source identities, and agreement of the two ordinal-control resolvers. Checks use explicit failures rather than relying on Python assertions.
2. The archived runtime receipts identify the fresh full Primary replay and the fresh Compatibility runtime replay. These receipts are evidence of those executions; the compact comparison checker does not rederive their full runtime or logical calibration.
3. In `source_pruning`, run `python3 -B check_pruning.py` and its `-O` counterpart. These replay the three source edits and their conditional control-flow obligations, not a smaller binary construction.
4. Search receipts retain inputs, family restrictions, depth, and limits. Only completed runs are counted; zero results are not lower bounds.

In `report`, `bash build_pdf.sh` compiles `report.tex` in two passes. Inherited licenses and attribution remain with the packets.

9.2 The offset simulation principle

For a deleted state q represented by r at offset k, use the boundary map $F(q, h, T) = (r, h + k, T)$ and $F(s, h, T) = (s, h, T)$ otherwise. An ordinary retained row entering q changes its displacement from d to $d + k$; legality requires $d + k \in \{-1, 1\}$. At $k = \pm 2$, all such entrances must point the opposite way. If every exceptional row has a positive, nonhalting, complete-tape macro to the image of its old successor, induction proves halting equivalence just as in Theorem 4.1. For $q = 0$, translating the homogeneous blank tape permits initial head 0, but every later runtime occurrence of q still needs its macro proof. No successful offset candidate is claimed.

9.3 Open constructive routes

Three precise extensions remain open; none is promised to save a state. First, the dead-loop deletion needs new sharing or allocation, since the fixed AST edit costs three DAG nodes. Second, simulation boundaries could permit controlled tape differences or phase-dependent offsets, beyond the present complete-tape and constant-offset templates. Third, a richer source-boundary guard domain could preserve correlations lost in seven- and nine-bit windows, especially inside the 25 conservatively opaque engine states.

Any advance must supply an actual total table of at most 207 working states, including initialization, with a sound boundary invariant, progress and halt reflection, and the retained proof-search calibration. A new table requires its own validation; relabeling a 208-state table or citing an unrelated certificate cannot close this obligation. The target remains unmet.

References and provenance

1. Vladimir Reshetnikov, ProveIt repository, incoming snapshot at commit `ef5a23e04b0285e321ceec5f4b329223cee0ab57`. `https://github.com/VladimirReshetnikov/ProveIt/tree/ef5a23e04b0285e321ceec5f4b329223cee0ab57/docs/incoming`. The Conditional Context, Entry Context, and History archives each supply a Primary and a Compatibility table, reports, and executable evidence.

2. ProveIt late incoming snapshot at commit `34aaab66fc5dc834863ed00ea84823c93b8c5b1e`. `https://github.com/VladimirReshetnikov/ProveIt/tree/34aaab66fc5dc834863ed00ea84823c93b8c5b1e/docs/incoming`. The plain U208 archive adds one table and the research archive adds two.

3. Conditional Context report, `paper/local_proof.tex` and `paper/calibration.tex`. Sources for the full five-state invariant, finite block proof, and explicit inherited unary-theory calibration. The Entry Context and History reports give corresponding presentations.

4. This investigation, `comparison/records.json`, `archive_identities.json`, `late_archive_identities.json`, and `check.py`. Exact table/source identities, graph maps, and comparison checker; normal and optimized receipts both report nine tables in two classes.

5. This investigation, `comparison/receipts/conditional_full.json` and `alternative_runtime.json`. Fresh representative runtime receipts; `entry_quick.json` and `history_quick.json` record distinct quick release replays with their narrower scope.

6. This investigation, `source_pruning/README.md`, `check_pruning.py`, and `expected.json`. Conditional closed-loop pruning and exact retained-edge transport. The separate reachable-source analysis supplies the assumed guards, not this compact checker itself.

7. This investigation, `local_search`, `synthesis`, and `boundary`. Search implementations, input identities, exact run receipts, and the projection argument in `PROJECTION.md`.

Archive identities

The following SHA-256 values identify the downloaded ZIP bytes, independently of names, native labels, or PDF metadata.

Conditional Context
`6f23aaca025984a9a3561c7cf352cbde069096713269d8ace9b8c0e305f3b251`

Entry Context
`00b0a699606a654bcc8b2079932ec3c6c7303b0adc019e548db2b12d62eb4868`

History
`2479f562ba30b68bcabcc9ab481cdcb20da2a74476bff132a99382a6669a88aa`

Late plain U208
`0662a968902f231ac2483bbd424f80b8f86e6fe1e04bbe97b6f25c634aa532f9`

Late research U208
`33666892d7e729084edb5aa6e2b5e07d0117adbb63b0f46baff5bbbdf1da3d09`